

Projeto Cartão - Cliente x Instituição

Trabalho I – Máquina de Estados | Inteligência Artificial e Ilusão de Inteligência em Jogos
Pontifícia Universidade Católica do Paraná – Prof.^a Marina de Lara
Gabriel Furlan | Gabriel Dias | Arthur Polak – Repositório: github.com/GabrielDyas/Projeto-Cartao

1. Visão geral do cenário

O projeto simula, dia após dia, a relação entre um **cliente** e a **instituição financeira** que emitiu o seu cartão de crédito. Cada iteração do laço principal (classe Run) representa um dia: a data avança com `ControleDeTempo.passarUmDia()` e, em seguida, o cliente age e depois a instituição age. O cliente decide se apenas passeia ou se sai para comprar; ao comprar, um produto aleatório do arquivo `_produtos.txt` (20 itens, de R\$ 60 a R\$ 3.500) é sorteado e fica registrado no cartão como compra pendente. A instituição percebe a compra pendente, compara o valor do produto com o limite disponível do cartão (inicialmente R\$ 10.000,00), aprova ou nega a compra, registra o resultado na fatura e, no dia seguinte, comunica a resposta. No dia 5 de cada mês (constante `FECHAMENTO_FATURA`) o cliente "fecha a fatura", e o sistema lista todas as compras da fatura com a respectiva situação, limpa a fatura e restabelece o limite disponível.

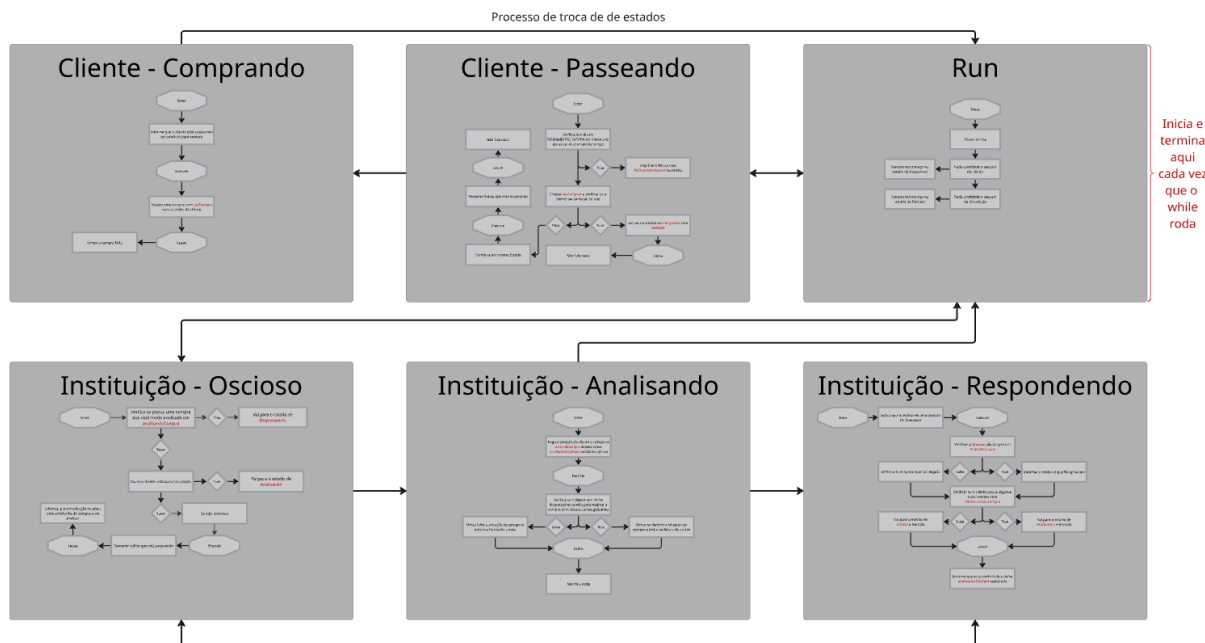
Os dois agentes não chamam métodos um do outro: eles se comunicam por meio do objeto compartilhado `Cartao` (campos `compra`, `Fatura` e `limites`) e de flags internas. Assim, o estado `Analisando` da instituição só é iniciado quando o cliente, no estado `Comprando`, deixa uma compra pendente no cartão, e o limite que o cliente exibe depende das aprovações feitas pela instituição – esta é a **comunicação entre agentes** (bônus do enunciado). Mapeamento com o enunciado: **Agente A = Instituição** (3 estados: `Ocioso`, `Analisando`, `Respondendo`) e **Agente B = Cliente** (2 estados: `Passeando`, `Comprando`). A duração é definida pela variável contador de `Run.java`: como o laço decrementa antes de simular, `contador = N` simula `N - 1` dias (valor atual: `6 → 5` dias).

2. Visão geral de cada agente

Cliente (Agente B) – classe `Cliente`, com os estados `Passeando` e `Comprando`. É o agente que dá início à interação: a cada dia o `Run` o coloca em `Passeando`; ao entrar nesse estado ele verifica se é o dia de fechamento da fatura, imprime o limite disponível e executa o teste de compra (um sorteio de 0 a 99; se o valor for menor que 50, ou seja, 50% de chance, o cliente passa para `Comprando`). Em `Comprando`, o cliente pede ao cartão um produto aleatório e o deixa como compra pendente; ao sair do estado, a compra pendente é limpa. O cliente nunca decide o resultado da compra: apenas a gera.

Instituição (Agente A) – classe `Instituicao`, com os estados `Ocioso`, `Analisando` e `Respondendo`. É o agente que concentra a regra de negócio: em `Ocioso` ela aguarda; ao detectar uma compra pendente no cartão do cliente vai para `Analisando`, onde valida o valor contra o limite disponível (aprova e debita o limite, ou nega) e grava a compra na fatura; no ciclo seguinte vai para `Respondendo`, onde informa se a compra foi aprovada ou negada e decide se há outra compra a analisar (volta a `Analisando`) ou se retorna a `Ocioso`. Por isso a resposta ao cliente tem sempre um dia de atraso em relação à compra.

3. Diagrama de estados por agente



Inicia e termina aqui cada vez que o while roda

Figura 1 – "Processo de troca de estados": visão geral da estrutura. Linha de cima: Cliente (`Comprando` e `Passeando`) e `Run`; linha de baixo: Instituição (`Ocioso`, `Analisando` e `Respondendo`). Cada caixa mostra o fluxo de `enter`, `execute` e `leave` de um estado; as setas ligam os estados ao `Run`, que a cada dia recoloca o `Cliente` em `Passeando` e a `Instituição` em `Ocioso`.

As Figuras 2 a 4 mostram cada bloco da Figura 1 separadamente, com as ações de `enter`, `execute` e `leave` e as decisões de cada estado.

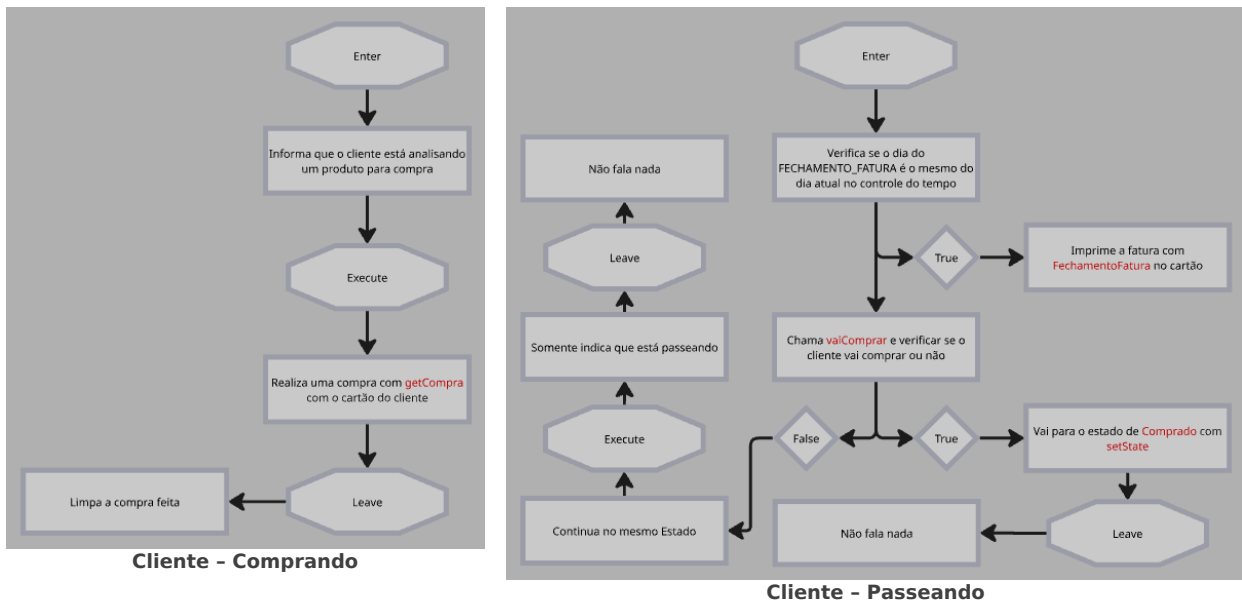


Figura 2 - Fluxo dos estados do Cliente (Agente B).

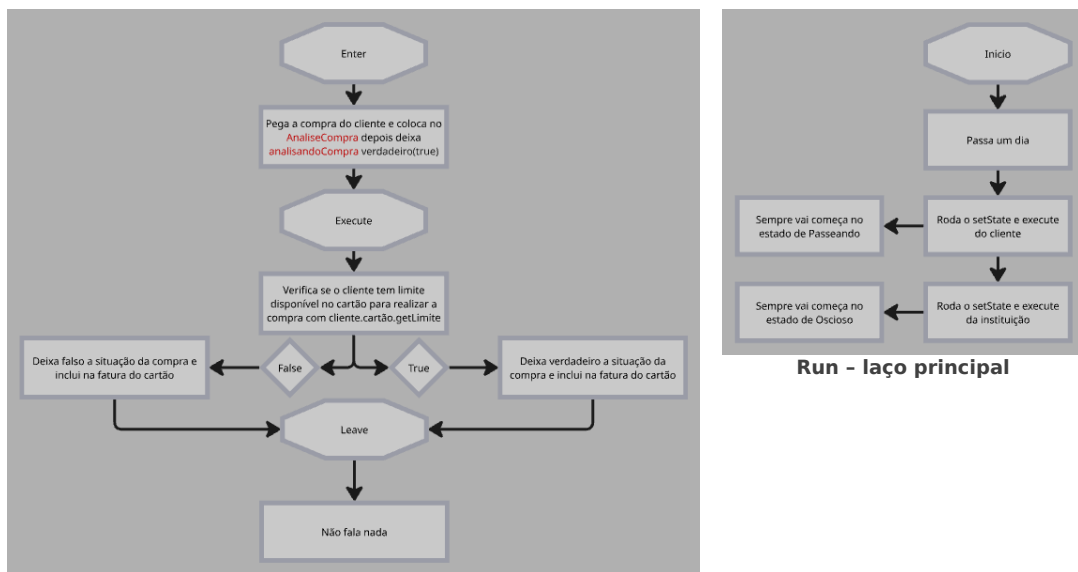


Figura 3 - Estado Analisando da Instituição (Agente A) e laço principal (Run), que a cada dia passa um dia e recoloca os agentes em Passeando e Ocioso.

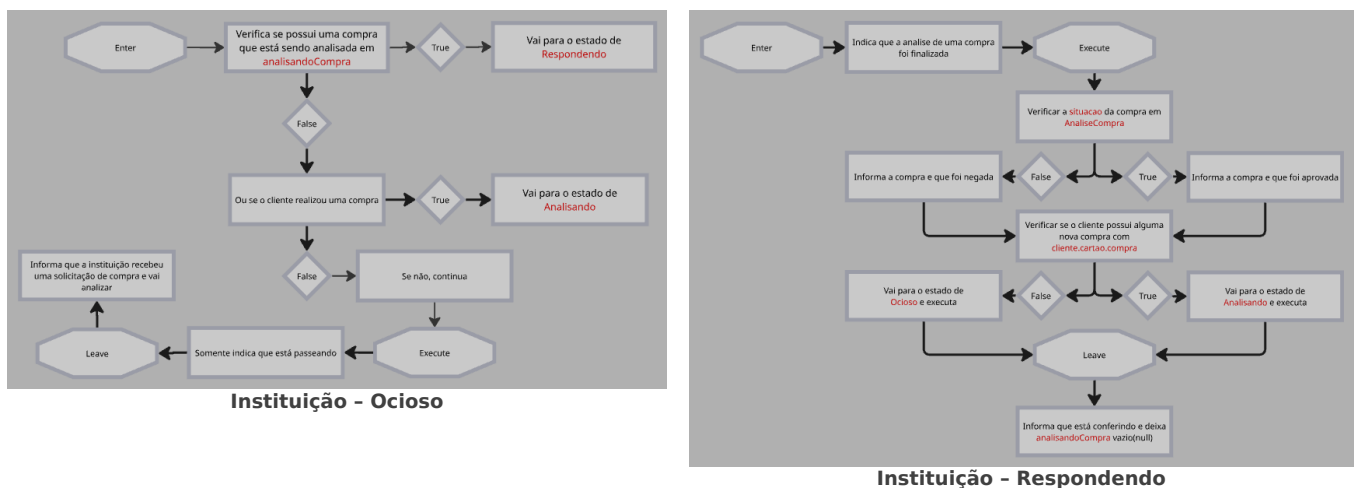


Figura 4 - Estados Ocioso e Respondendo da Instituição (Agente A).

4. Tabela de regras por agente

A única fonte de aleatoriedade do sistema é `vaiComprar()` (50% de chance de compra), que modela um consumidor que nem sempre compra; todas as demais transições são determinísticas e dependem apenas de flags e da compra pendente no cartão.

Cliente (Agente B)

Estado	Ação de entrada (enter)	Ação de execução (execute)	Ação de saída (leave)	Transições
Passeando	Imprime separador. Se o dia do mês = 5: imprime "Dia de fechamento da fatura" e chama <code>cartao.FechamentoFatura()</code> (lista a fatura, a limpa e restabelece o limite). Imprime o limite disponível (<code>logCliente()</code>). Sorteia <code>vaiComprar()</code> : se verdadeiro imprime "saindo para comprar" e troca para Comprando; senão imprime "saindo para passear".	Imprime "Cliente está passeando". (Só ocorre nos dias em que não houve troca para Comprando.)	Nenhuma ação.	→ Comprando se <code>nextInt(100) < 50</code> (feito no próprio enter). Caso contrário permanece em Passeando.
Comprando	Faz <code>comprou = true</code> e imprime "Analisando produto para compra...".	Faz <code>cartao.compra = cartao.getCompra()</code> (sorteia produto do arquivo) e imprime nome e valor: "Cliente está comprando: ...".	Faz <code>cartao.compra = null</code> (limpa a compra pendente).	→ Passeando no início do dia seguinte (o Run chama <code>setState(new Passeando())</code>).

Instituição (Agente A)

Estado	Ação de entrada (enter)	Ação de execução (execute)	Ação de saída (leave)	Transições
Ocioso	Não imprime nada; apenas testa as flags: se <code>analizandoCompra</code> → troca para Respondendo; senão, se <code>cartao.compra != null</code> → troca para Analisando.	Imprime "Instituição está ociosa."	Imprime "Solicitação recebida, iniciando análise." (é impresso sempre que Ocioso é deixado).	→ Respondendo se <code>analizandoCompra = true</code> ; → Analisando se <code>analizandoCompra = false</code> e <code>compra != null</code> ; senão permanece em Ocioso.
Analisando	Imprime "Iniciando análise de compra X no valor de R\$ Y..."; guarda a compra em <code>AnaliseCompra</code> e faz <code>analizandoCompra = true</code> .	Imprime "Analisando compra...". Se <code>limiteDisponivel < valor: situacao = false</code> e adiciona à fatura. Senão: <code>situacao = true</code> , adiciona à fatura e subtrai o valor de <code>LimiteDisponivel</code> .	Nenhuma ação.	→ Ocioso no início do dia seguinte (Run chama <code>setState(new Ocioso())</code>), que redireciona para Respondendo pois <code>analizandoCompra = true</code> .
Respondendo	Imprime "Análise de compra finalizada."	Imprime "Compra de(a) X no valor de R\$ Y aprovada/negada." Em seguida: se <code>cartao.compra != null</code> → vai para Analisando e executa; senão → vai para Ocioso e executa.	Imprime "Conferindo..." e faz <code>analizandoCompra = false</code> .	→ Analisando se <code>cartao.compra != null</code> (nova compra do dia); → Ocioso se <code>cartao.compra == null</code> .

5. Descrição das variáveis e limiares por agente

Cliente (Agente B)

Nome	Tipo	Intervalo	Valor inicial	Uso / limiar de troca de estado
<code>currentState</code>	<code>State<Cliente></code>	Passeando Comprando	null (Run define Passeando)	Estado atual; alterado por <code>setState()</code> (chama leave do antigo e enter do novo).
<code>comprou</code>	<code>boolean</code>	true false	false	Marcada como true no enter de Comprando. Não é lida por outra classe nem reiniciada.
<code>cartao</code>	<code>Cartao</code>	-	novo <code>Cartao(new Limites())</code>	Cartão do cliente; objeto compartilhado com a Instituição.
<code>instituicao</code>	<code>Instituicao</code>	-	null (Run atribui)	Referência ao outro agente.
<code>chance (local)</code>	<code>int</code>	0 a 99	sorteio a cada dia	Limiar: <code>chance < 50</code> → Comprando (probabilidade de 50%).

Instituição (Agente A)

Nome	Tipo	Intervalo	Valor inicial	Uso / limiar de troca de estado
<code>currentState</code>	<code>State<Instituicao></code>	Ocioso Analisando Respondendo	null (Run define Ocioso)	Estado atual.
<code>analizandoCompra</code>	<code>boolean</code>	true false	false	true entre o enter de Analisando e o leave de Respondendo; decide Ocioso → Respondendo.
<code>AnaliseCompra</code>	<code>Produto</code>	-	null	Compra em análise; usada em Analisando e Respondendo.
<code>cliente</code>	<code>Cliente</code>	-	recebido no construtor	Referência ao outro agente (lê <code>cartao.compra</code> e altera o limite).
Limiar de aprovação	<code>float</code>	0 a 10000	-	Compra negada se <code>LimiteDisponivel < valor</code> ; aprovada se <code>LimiteDisponivel ≥ valor</code> (valor igual ao limite é aprovado).

Variáveis compartilhadas (objetos de apoio)

Nome	Tipo	Intervalo	Valor inicial	Descrição
Limites.LIMITE_TOTAL	final float	constante	10000	Limite total do cartão.
Limites.LimiteDisponivel	float	0 a 10000	10000	Diminui do valor de cada compra aprovada; volta a 10000 no fechamento da fatura.
Cartao.FECHAMENTO_FATURA	final int	dia do mês (1-31)	5	Quando getDayOfMonth() == 5 o cliente fecha a fatura.
Cartao.compra	Produto	-	null	Compra pendente: escrita pelo Cliente, lida pela Instituição.
Cartao.Fatura	List<Produto>	0 a N compras	lista vazia	Compras analisadas (aprovadas e rejeitadas); esvaziada em FechamentoFatura().
Produto.nome / valor / dataCompra / situacao	String / double / LocalDate / boolean	valor: R\$ 60 a R\$ 3.500	lido de _produtos.txt; situacao = false	Produto sorteado; situacao: true = aprovada, false = negada.
ControleDeTempo.dia	static LocalDate	data	LocalDate.now()	Avança 1 dia por iteração do laço.
Run.contador	int	≥ 1	6	Controla a duração: simula contador – 1 dias.

6. Estrutura do código

Todo o código usa apenas Java padrão (java.time, java.nio.file, java.util). O padrão State está em State<T> e AbstractState<T>; cada estado concreto sobrescreve enter, execute e leave. A Figura 5 traz os diagramas do grupo com as variáveis e funções de cada classe, e a tabela resume o papel de cada arquivo.

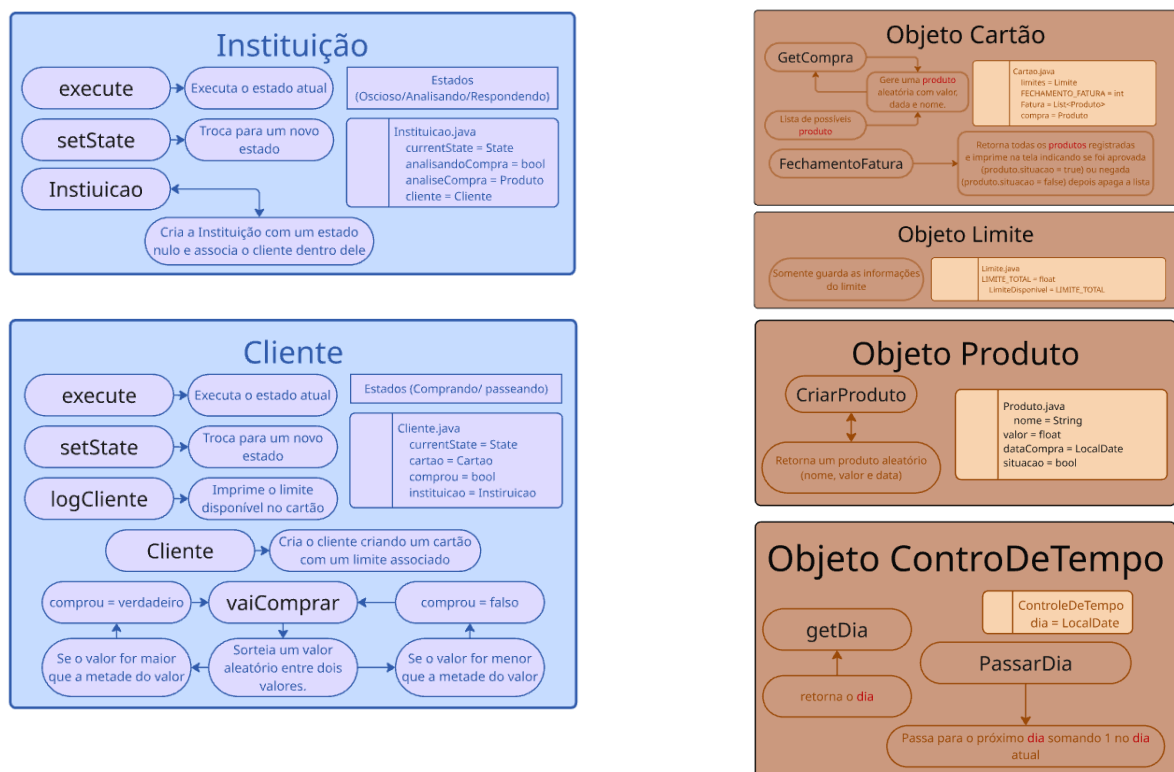


Figura 5 – Diagramas das classes: Instituição, Cartão/Limite, Cliente e Produto/ControleDeTempo. Nota: no código, vaiComprar() retorna true quando o sorteio (0 a 99) é **menor que 50**, e o campo comprou é marcado em Comprando. enter; o tipo de Produto.valor é double.

Arquivo / classe	Papel no projeto
Run/Run.java	Laço da simulação: cria Cliente e Instituição, avança o dia, recoloca o Cliente em Passeando e a Instituição em Ocioso e chama execute() de cada um.
Objetos/State.java, AbstractState.java	Interface do padrão State (enter, execute, leave) e classe base abstrata dos estados.
cliente/Cliente.java	Contexto do Agente B: estado atual, cartão e flags; setState() (leave → troca → enter), execute(), vaiComprar() e logCliente().
cliente/states/Passeando, Comprando	Estados concretos do Cliente (seção 4).
instituicao/Instituicao.java	Contexto do Agente A: estado atual, compra em análise e referência ao Cliente; setState() e execute().
instituicao/States/Ocioso, Analisando, Respondendo	Estados concretos da Instituição (seção 4).
Objetos/Cartao.java, Limites.java	Cartão compartilhado (limites, fatura, compra pendente, dia de fechamento, getCompra() e FechamentoFatura(), que lista e limpa a fatura e repõe o limite) e limite (total R\$ 10.000 e disponível).

Arquivo / classe	Papel no projeto
Objetos/Produto.java, _produtos.txt	Produto/compra (nome, valor, data, situação); CriarProduto() lê o catálogo de 20 produtos e sorteia um item.
Objetos/ControleDeTempo.java	Relógio simulado: data atual e passarUmDia().

7. Resultados esperados (exemplos de logs de transição)

Trecho de uma execução real da versão atual do código (dias 04 a 07/10). Para mostrar o fechamento de fatura e uma compra negada em poucos dias, a data inicial foi 02/10 e o limite inicial R\$ 100,00 (valores ajustados só para a demonstração); com os valores padrão a estrutura do log é a mesma. Como o sorteio é aleatório, cada execução produz um log diferente.

```

Dia: 2026-10-04
-----
Limite disponível: R$100.00
Cliente está saindo para comprar.
Analisando produto para compra...
Cliente está comprando: Placa de vídeo RTX no valor de R$3500.00
-----
Solicitação recebida, iniciando análise.
Solicitação recebida, iniciando análise.
Iniciando análise de compra Placa de vídeo RTX no valor de R$3500.00...
Analisando compra...

Dia: 2026-10-05
-----
Dia de fechamento da fatura.
Fatura fechada e limite restabelecido!
Situação das compras:
Compra: Placa de vídeo RTX | Valor: R$3500.00 | Situação: Rejeitada
-----
Limite disponível: R$10000.00
Cliente está saindo para passear.
Cliente está passeando
-----
Solicitação recebida, iniciando análise.
Análise de compra finalizada.
Compra de(a) Placa de vídeo RTX no valor de R$3500.00 negada.Conferindo...
Instituição está ociosa.

Dia: 2026-10-06
-----
Limite disponível: R$10000.00
Cliente está saindo para comprar.
Analisando produto para compra...
Cliente está comprando: Controle para videogame no valor de R$250.00
-----
Solicitação recebida, iniciando análise.
Solicitação recebida, iniciando análise.
Iniciando análise de compra Controle para videogame no valor de R$250.00...
Analisando compra...

Dia: 2026-10-07
-----
Limite disponível: R$9750.00
Cliente está saindo para passear.
Cliente está passeando
-----
Solicitação recebida, iniciando análise.
Análise de compra finalizada.
Compra de(a) Controle para videogame no valor de R$250.00 aprovada.Conferindo...
Instituição está ociosa.

```

Como ler as transições no log

Linha do log	Evento de estado	Linha do log	Evento de estado
Cliente está saindo para comprar.	Passeando.enter → setState(Comprando)	Solicitação recebida, iniciando análise.	Ocioso.leave
Analisando produto para compra...	Comprando.enter	Iniciando análise de compra ...	Analisando.enter
Cliente está comprando: ...	Comprando.execute	Analisando compra...	Analisando.execute
Cliente está passeando	Passeando.execute	Análise de compra finalizada.	Respondendo.enter
Dia de fechamento da fatura.	Passeando.enter (dia 5)	Compra de(a) ... aprovada/negada.	Respondendo.execute
Instituição está ociosa.	Ocioso.execute	Conferindo...	Respondendo.leave

No log: a compra do dia 04 (Placa de vídeo, R\$ 3.500) é analisada no mesmo dia, mas só é respondida no dia 05 ("negada"), por ser maior que o limite disponível (R\$ 100). Ainda no dia 05, o fechamento da fatura lista a compra como rejeitada e restabelece o limite para R\$ 10.000, o que permite aprovar a compra do dia 06 (Controle, R\$ 250).

Observação: para a formatação deste documento e a organização do repositório no GitHub foi utilizada a IA **Claude** (Anthropic). Para o desenvolvimento da programação foi utilizada a IA **Gemini** (Google), como suporte ao entendimento da estrutura do código.